

Réunion du 24 Juin 2025

Une première série de tests sur la réduction de modèles par POD et NN-augmented POD

Sofiane Ezzehi

École Nationale des Ponts et Chaussées

24 Juin 2025



ÉCOLE NATIONALE DES
PONTS
ET CHAUSSEES



Plan

1. Cas test : Équation de diffusion avec fracture
2. Résolution haute-fidélité
3. Cas Affine
4. Cas Non-affine I
5. Cas Non-affine II
6. Kolmogorov n -width et POD
7. NN-augmented POD



Cas test : Équation de diffusion avec fracture

Équation de diffusion sur $\Omega = (0, 100) \times (0, 100)$ paramétrée par $\mu \in \mathcal{P} \subset \mathbb{R}^p$,

$$\operatorname{div}(D_\mu(x, y) \nabla u_\mu(x, y)) = 0, \quad (x, y) \in \Omega,$$

avec conditions aux bords de Dirichlet,

$$u_\mu(x, y) = \bar{u}(x, y), \quad (x, y) \in \partial\Omega,$$

où $\bar{u}(x, y)$ est une fonction sur $\partial\Omega$ indépendante de μ .



Paramètres du problème

On prend le profil de diffusion suivant,

$$D_{\mu}(x, y) = D_{\text{back}} \cdot (\sigma(-ax - y + (x_0 - w)) + \sigma(ax + y - (x_0 + w))) \\ + D_{\text{fract}} \cdot \sigma(-ax - y + (x_0 - w)) \cdot \sigma(ax + y - (x_0 + w)) \quad (1)$$

où $\mu = (x_0, w, a, D_{\text{back}}, D_{\text{fract}})$ est un vecteur de paramètres et σ est la fonction de Heaviside.

Paramètre	Description
x_0	Centre de la fracture
w	Largeur de la fracture
a	Pente de la fracture
D_{back}	Coefficient de diffusion dans le fond
D_{fract}	Coefficient de diffusion dans la fracture

Table: Description des paramètres du profil de diffusion.



Exemple

On prend $\bar{u}(x, y)$ comme suit,

$$\bar{u}(x, y) = \begin{cases} 4 & \text{si } x \leq 10 \text{ et } y \geq 90, \\ 1 & \text{si } y \leq 10, \\ 0 & \text{sinon.} \end{cases} \quad (2)$$

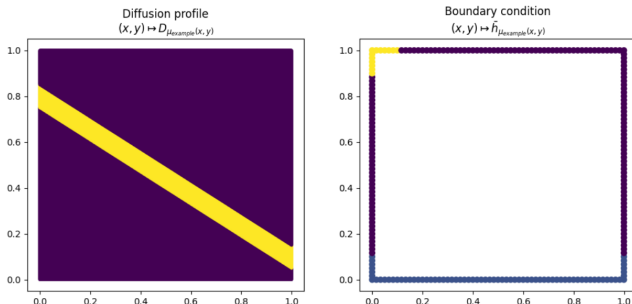


Figure: The x and y axes are normalized to $[0, 1]$. The diffusion profile is defined by the parameters: $D_{\text{back}} = 1e^{-6}$, $D_{\text{fract}} = 1e^{-3}$, $x_0 = 80$, $w = 5$ et $a = 60$.



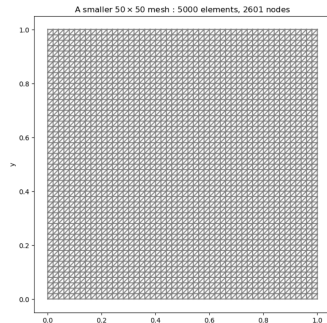
Plan

1. Cas test : Équation de diffusion avec fracture
2. Résolution haute-fidélité
3. Cas Affine
4. Cas Non-affine I
5. Cas Non-affine II
6. Kolmogorov n -width et POD
7. NN-augmented POD



Résolution du problème haute-fidélité

Caractéristique	Valeur
Résolution du maillage	200×200
Type de cellules	Triangles
Nombre d'éléments (cellules)	80 000
Nombre de sommets (nœuds)	40 401
Type d'éléments finis	Lagrange
Degré des éléments finis	2
Nombre de degrés de liberté	160 801
Temps de résolution moyen	3.27 s



Plan

1. Cas test : Équation de diffusion avec fracture
2. Résolution haute-fidélité
3. Cas Affine
4. Cas Non-affine I
5. Cas Non-affine II
6. Kolmogorov n -width et POD
7. NN-augmented POD



Génération des snapshots

Type	Paramètres	Intervalles	Nb. snap
Affine	$(D_{\text{back}}, D_{\text{fract}})$	$[10^{-6}, 10^{-4}] \times [10^{-3}, 10^{-2}]$	10×10
Non-affine I	$(D_{\text{fract}}, x_0, w)$	$[10^{-3}, 10^{-2}] \times [30, 90] \times [1, 10]$	$10 \times 10 \times 10$
Non-affine II	(x_0, w, a)	$[30, 90] \times [1, 10] \times [10, 100]$	$10 \times 10 \times 10$

Table: Snapshots générés pour les trois applications.

Utilisation des snapshots	Proportion
Entraînement (POD)	70% tirés aléatoirement
Test	30% restants

Table: Répartition des snapshots pour l'entraînement et le test.



Décroissance de l'énergie - Cas Affine

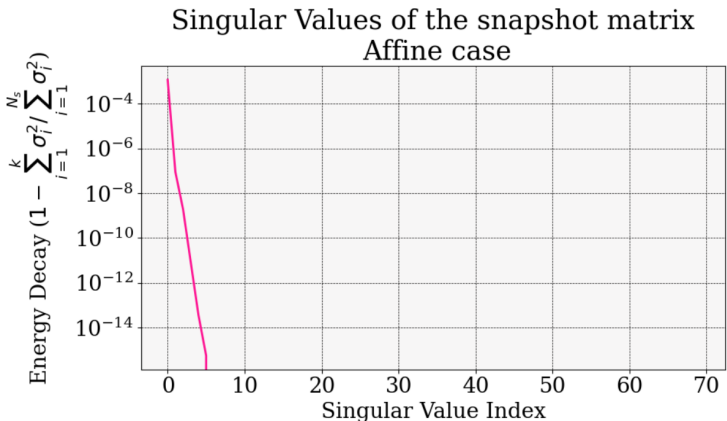
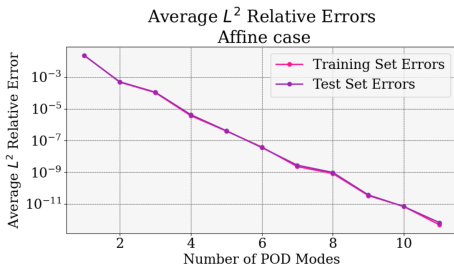


Figure: Décroissance de l'énergie des 70 modes d'entraînement pour le cas affine.



Résultats de la POD - Cas Affine

- Comme attendu, la POD est en mesure de capturer, avec très peu de modes (≤ 10), la dynamique du problème affine a des précisions de l'ordre de 10^{-12} .



- Le temps de calcul online est très faible grâce à la condition de séparabilité qui est ici vérifiée.

Nb modes	Temps de calcul moyen (s)		
	avec séparabilité	sans séparabilité	HDM
10	0.004	0.2	3.2



En pratique : calculs avec et sans séparabilité

Le système HDM à inverser est,

$$A_{\mu}^N y_{\mu}^N = L_{\mu}^N,$$

avec $A_{\mu}^N \in \mathbb{R}^{N \times N}$ et $L_{\mu}^N \in \mathbb{R}^N$.



En pratique : calculs avec et sans séparabilité

Le système HDM à inverser est,

$$A_{\mu}^N y_{\mu}^N = L_{\mu}^N,$$

avec $A_{\mu}^N \in \mathbb{R}^{N \times N}$ et $L_{\mu}^N \in \mathbb{R}^N$.

Soit $V_r \in \mathbb{R}^{N \times r}$ la base POD de rang $r \ll N$ construite à partir des snapshots.

On introduit

$$A_{\mu}^r = V_r^T A_{\mu}^N V_r \in \mathbb{R}^{r \times r}, \quad L_{\mu}^r = V_r^T L_{\mu}^N \in \mathbb{R}^r.$$



En pratique : calculs avec et sans séparabilité

Le système HDM à inverser est,

$$A_{\mu}^N y_{\mu}^N = L_{\mu}^N,$$

avec $A_{\mu}^N \in \mathbb{R}^{N \times N}$ et $L_{\mu}^N \in \mathbb{R}^N$.

Soit $V_r \in \mathbb{R}^{N \times r}$ la base POD de rang $r \ll N$ construite à partir des snapshots.

On introduit

$$A_{\mu}^r = V_r^T A_{\mu}^N V_r \in \mathbb{R}^{r \times r}, \quad L_{\mu}^r = V_r^T L_{\mu}^N \in \mathbb{R}^r.$$

Pour résoudre le problème réduit, il faut alors inverser,

$$A_{\mu}^r y_{\mu}^r = L_{\mu}^r.$$



En pratique : calculs avec et sans séparabilité

Cependant, cela implique qu'il faut assembler, en phase online, les matrices A_μ^r et L_μ^r pour chaque μ , ce qui implique **des multiplications de matrices dépendant de N** .



En pratique : calculs avec et sans séparabilité

Cependant, cela implique qu'il faut assembler, en phase online, les matrices A_μ^r et L_μ^r pour chaque μ , ce qui implique **des multiplications de matrices dépendant de N** .

Sans condition de séparabilité, on doit assembler $A_\mu^N = V_r^T A_\mu^N V_r$ et $L_\mu^N = V_r^T L_\mu^N$ pour chaque μ . $\Rightarrow$ Coûteux et potentiellement plus lent que la résolution HDM.



En pratique : calculs avec et sans séparabilité

Cependant, cela implique qu'il faut assembler, en phase online, les matrices A_μ^r et L_μ^r pour chaque μ , ce qui implique **des multiplications de matrices dépendant de N** .

Sans condition de séparabilité, on doit assembler $A_\mu^N = V_r^T A_\mu^N V_r$ et $L_\mu^N = V_r^T L_\mu^N$ pour chaque μ . $\Rightarrow$ Coûteux et potentiellement plus lent que la résolution HDM.

Avec condition de séparabilité, $A_\mu^r = \mu_1 A_1 + \mu_2 A_2$. $\Rightarrow$ **(1)** On évalue $V^T A_1 V$ et $V^T A_2 V$ une seule fois **en phase offline**. **(2)** Puis, **en phase online**, on les combine pour tout nouveau μ . De même pour $L_\mu^r = \mu_1 L_1 + \mu_2 L_2$.



Plan

1. Cas test : Équation de diffusion avec fracture
2. Résolution haute-fidélité
3. Cas Affine
4. Cas Non-affine I
5. Cas Non-affine II
6. Kolmogorov n -width et POD
7. NN-augmented POD



Génération des snapshots

Type	Paramètres	Intervalles	Nb. snap
Affine	$(D_{\text{back}}, D_{\text{fract}})$	$[10^{-6}, 10^{-4}] \times [10^{-3}, 10^{-2}]$	10×10
Non-affine I	$(D_{\text{fract}}, x_0, w)$	$[10^{-3}, 10^{-2}] \times [30, 90] \times [1, 10]$	$10 \times 10 \times 10$
Non-affine II	(x_0, w, a)	$[30, 90] \times [1, 10] \times [10, 100]$	$10 \times 10 \times 10$

Table: Snapshots générés pour les trois applications.

Utilisation des snapshots	Proportion
Entraînement (POD)	70% tirés aléatoirement
Test	30% restants

Table: Répartition des snapshots pour l'entraînement et le test.



Décroissance de l'énergie - Cas Non-affine I

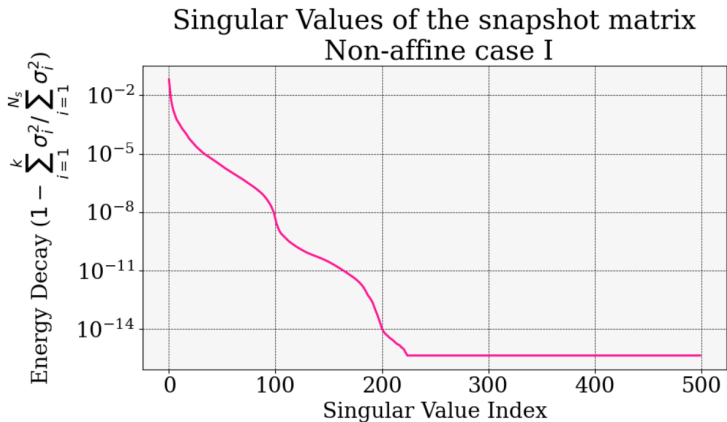


Figure: Décroissance de l'énergie des 500 premiers modes d'entraînement pour le cas non-affine I.



Résultats de la POD - Cas Non-affine I

Average Relative L^2 Errors and Computation Time vs Number of Modes
Non-Affine case I

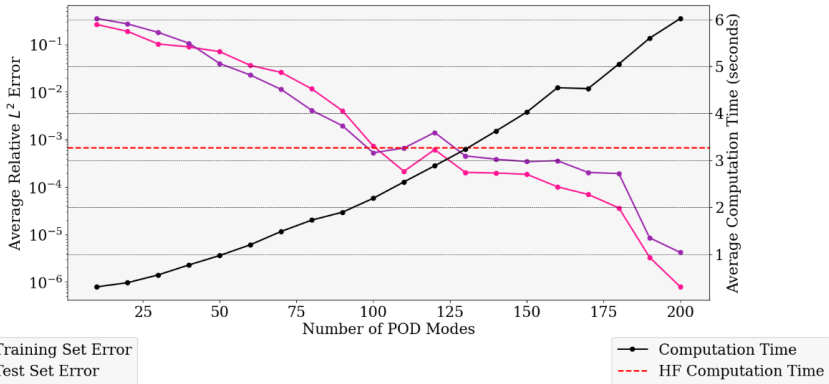


Figure: Gauche : Erreur l^2 réalisée sur les snapshots d'entraînement et test pour le cas non-affine I. Droite : Temps de calcul Online.



Plan

1. Cas test : Équation de diffusion avec fracture
2. Résolution haute-fidélité
3. Cas Affine
4. Cas Non-affine I
5. Cas Non-affine II
6. Kolmogorov n -width et POD
7. NN-augmented POD



Génération des snapshots

Type	Paramètres	Intervalles	Nb. snap
Affine	$(D_{\text{back}}, D_{\text{fract}})$	$[10^{-6}, 10^{-4}] \times [10^{-3}, 10^{-2}]$	10×10
Non-affine I	$(D_{\text{fract}}, x_0, w)$	$[10^{-3}, 10^{-2}] \times [30, 90] \times [1, 10]$	$10 \times 10 \times 10$
Non-affine II	(x_0, w, a)	$[30, 90] \times [1, 10] \times [10, 100]$	$10 \times 10 \times 10$

Table: Snapshots générés pour les trois applications.

Utilisation des snapshots	Proportion
Entraînement (POD)	70% tirés aléatoirement
Test	30% restants

Table: Répartition des snapshots pour l'entraînement et le test.



Décroissance de l'énergie - Cas Non-affine II

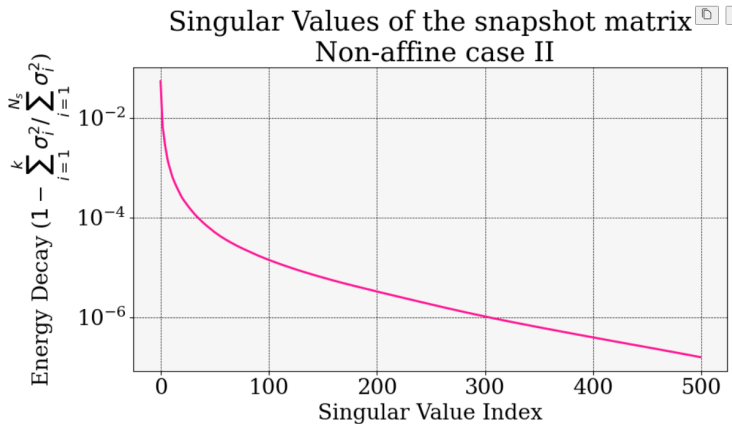


Figure: Décroissance de l'énergie des 500 premiers modes d'entraînement pour le cas non-affine II.



Résultats de la POD - Cas Non-affine II

Average Relative L^2 Errors and Computation Time vs Number of Modes
Non-Affine case II

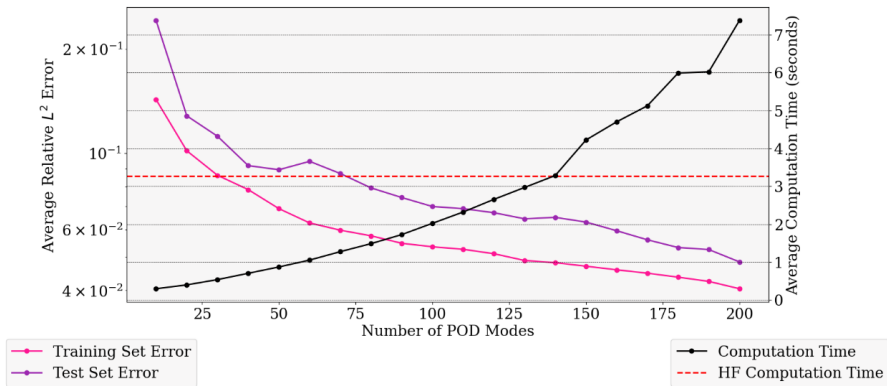


Figure: Gauche : Erreur l^2 réalisée sur les snapshots d'entraînement et test pour le cas non-affine II. Droite : Temps de calcul Online.



Quelques remarques

- Ces tests permettent d'avoir une idée du gain de temps qu'on peut obtenir **-sans hyperréduction-** en utilisant la POD classique pour nos problèmes.
- L'idée pour l'instant est de se concentrer sur la construction des ROMs avant d'implémenter des techniques d'hyperréduction. Si on se limite à peu de modes, on peut tout de même espérer des gains de temps significatifs.



Plan

1. Cas test : Équation de diffusion avec fracture
2. Résolution haute-fidélité
3. Cas Affine
4. Cas Non-affine I
5. Cas Non-affine II
6. Kolmogorov n -width et POD
7. NN-augmented POD



La Barrière de Kolmogorov

- La Kolmogorov n -width est une mesure de la capacité d'un espace de fonctions à être approché par des sous-espaces de dimension finie :

$$d_n(\mathcal{F}) = \inf_{\mathcal{G}_n} \sup_{f \in \mathcal{F}} \inf_{g_n \in \mathcal{G}_n} \|f - g_n\|,$$

où $\mathcal{F}$ est l'espace de fonctions, $\mathcal{G}_n$ un sous-espace de dimension n de $\mathcal{F}$, et $\|\cdot\|$ une norme sur cet espace.



La Barrière de Kolmogorov

- La Kolmogorov n -width est une mesure de la capacité d'un espace de fonctions à être approché par des sous-espaces de dimension finie :

$$d_n(\mathcal{F}) = \inf_{\mathcal{G}_n} \sup_{f \in \mathcal{F}} \inf_{g_n \in \mathcal{G}_n} \|f - g_n\|,$$

où $\mathcal{F}$ est l'espace de fonctions, $\mathcal{G}_n$ un sous-espace de dimension n de $\mathcal{F}$, et $\|\cdot\|$ une norme sur cet espace.

- La Kolmogorov n -width constitue une borne inférieure de l'erreur d'approximation par POD.



La Barrière de Kolmogorov

- La Kolmogorov n -width est une mesure de la capacité d'un espace de fonctions à être approché par des sous-espaces de dimension finie :

$$d_n(\mathcal{F}) = \inf_{\mathcal{G}_n} \sup_{f \in \mathcal{F}} \inf_{g_n \in \mathcal{G}_n} \|f - g_n\|,$$

où $\mathcal{F}$ est l'espace de fonctions, $\mathcal{G}_n$ un sous-espace de dimension n de $\mathcal{F}$, et $\|\cdot\|$ une norme sur cet espace.

- La Kolmogorov n -width constitue une borne inférieure de l'erreur d'approximation par POD.
- Une décroissance lente de l'énergie des modes (valeurs singulières) de la POD indique une grande Kolmogorov n -width, ce qui signifie que l'espace de fonctions est difficile à approximer par des sous-espaces de dimension finie.



Comment mitiger la Barrière de Kolmogorov ?

L'idée la plus simple est d'**altérer la base traditionnellement affine de la POD** en introduisant de la **non-linéarité**. Ces stratégies incluent :

- 1 La construction d'un espace d'approximation **affine par morceaux** en utilisant les bases locale.



Comment mitiger la Barrière de Kolmogorov ?

L'idée la plus simple est d'**altérer la base traditionnellement affine de la POD** en introduisant de la **non-linéarité**. Ces stratégies incluent :

- ① La construction d'un espace d'approximation **affine par morceaux** en utilisant les bases locale.
- ② La construction d'un espace d'approximation **quadratique** au lieu de linéaire.



Comment mitiger la Barrière de Kolmogorov ?

L'idée la plus simple est d'**altérer la base traditionnellement affine de la POD** en introduisant de la **non-linéarité**. Ces stratégies incluent :

- ① La construction d'un espace d'approximation **affine par morceaux** en utilisant les bases locale.
- ② La construction d'un espace d'approximation **quadratique** au lieu de linéaire.
- ③ La combinaison de ces deux approches précédentes pour obtenir un espace d'approximation **quadratique par morceaux**.



Comment mitiger la Barrière de Kolmogorov ?

L'idée la plus simple est d'**altérer la base traditionnellement affine de la POD** en introduisant de la **non-linéarité**. Ces stratégies incluent :

- ① La construction d'un espace d'approximation **affine par morceaux** en utilisant les bases locale.
- ② La construction d'un espace d'approximation **quadratique** au lieu de linéaire.
- ③ La combinaison de ces deux approches précédentes pour obtenir un espace d'approximation **quadratique par morceaux**.
- ④ **Méthode récente à la Charbel**: Introduire un terme de correction non-linéaire dans la POD, qui consiste à un **réseau de neurones** qui apprend à corriger les erreurs de la POD linéaire.



Plan

1. Cas test : Équation de diffusion avec fracture
2. Résolution haute-fidélité
3. Cas Affine
4. Cas Non-affine I
5. Cas Non-affine II
6. Kolmogorov n -width et POD
7. NN-augmented POD



NN-augmented POD

Classiquement, on construit une approximation de la solution,

$$u \approx \tilde{u} = u_0 + V_r y_r,$$

où $u \in \mathbb{R}^N$ est la solution HF, $u_0 \in \mathbb{R}^N$ est une solution de référence, $V_r \in \mathbb{R}^{N \times r}$ est la base POD et $y_r \in \mathbb{R}^r$ est un vecteur de coefficients solution du problème réduit (ROM).



NN-augmented POD

Classiquement, on construit une approximation de la solution,

$$u \approx \tilde{u} = u_0 + V_r y_r,$$

où $u \in \mathbb{R}^N$ est la solution HF, $u_0 \in \mathbb{R}^N$ est une solution de référence, $V_r \in \mathbb{R}^{N \times r}$ est la base POD et $y_r \in \mathbb{R}^r$ est un vecteur de coefficients solution du problème réduit (ROM).

Quand la Kolmogorov n -width décroît lentement, il faut r trop grand pour obtenir une bonne approximation, ce qui rend la ROM inutilisable (sans hyperréduction).

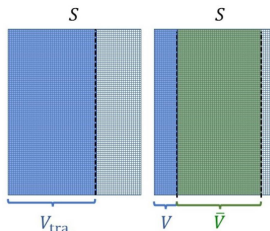


NN-augmented POD

L'idée de la méthode NN-augmented POD est de se limiter à un espace d'approximation de dimension $n \ll r$ et d'introduire un terme de correction non-linéaire pour apprendre l'information dans les $\bar{n} = r - n$ modes restants,

$$u \approx \tilde{u} = u_0 + V y + \bar{V} \mathcal{N}_\theta(y),$$

où $V_r = [V \mid \bar{V}]$, d'où $V \in \mathbb{R}^{N \times n}$ et $\bar{V} \in \mathbb{R}^{N \times \bar{n}}$, $y \in \mathbb{R}^n$ et $\mathcal{N}_\theta : \mathbb{R}^n \rightarrow \mathbb{R}^{\bar{n}}$ est un réseau de neurones paramétré par θ .



Construction of a ROB for a traditional PROM (left); and a PROM-ANN (right)



Entraînement du réseau de neurones

Pour l'entraînement, l'article propose d'utiliser les snapshots générés pour la POD,

Snapshots : $(u_l)_{l=1}^{N_s}$.



Entraînement du réseau de neurones

Pour l'entraînement, l'article propose d'utiliser les snapshots générés pour la POD,

$$\textbf{Snapshots} : (u_I)_{I=1}^{N_s}.$$

La projection de ces snapshots sur les bases V et $\overline{V}$ donne, pour $1 \leq I \leq N_s$,

$$\textbf{Projections} : V^T(u_I - u_0) = y_I, \quad \overline{V}^T(u_I - u_0) = \overline{y}_I.$$



Entraînement du réseau de neurones

Pour l'entraînement, l'article propose d'utiliser les snapshots générés pour la POD,

$$\textbf{Snapshots} : (u_l)_{l=1}^{N_s}.$$

La projection de ces snapshots sur les bases V et $\overline{V}$ donne, pour $1 \leq l \leq N_s$,

$$\textbf{Projections} : V^T(u_l - u_0) = y_l, \quad \overline{V}^T(u_l - u_0) = \overline{y}_l.$$

D'autre part,

$$u_l - u_0 \approx \tilde{u}_l - u_0 = Vy_l + \overline{V}\mathcal{N}_\theta(y_l).$$



Entraînement du réseau de neurones

Pour l'entraînement, l'article propose d'utiliser les snapshots générés pour la POD,

$$\textbf{Snapshots} : (u_l)_{l=1}^{N_s}.$$

La projection de ces snapshots sur les bases V et $\bar{V}$ donne, pour $1 \leq l \leq N_s$,

$$\textbf{Projections} : V^T(u_l - u_0) = y_l, \quad \bar{V}^T(u_l - u_0) = \bar{y}_l.$$

D'autre part,

$$u_l - u_0 \approx \tilde{u}_l - u_0 = Vy_l + \bar{V}\mathcal{N}_\theta(y_l).$$

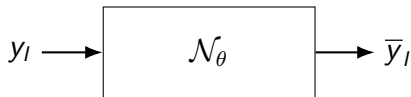
En utilisant que $V^T V = I_n$, $\bar{V}^T \bar{V} = I_{\bar{n}}$ et $\bar{V}^T V = 0$ **et** en omettant l'approximation $\approx$ (comme fait implicitement dans l'article), on peut écrire,

$$\bar{V}^T(u_l - u_0) = \mathcal{N}_\theta(y_l).$$



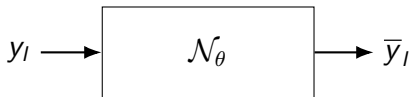
Entraînement du réseau de neurones

On veut donc entraîner le réseau de neurones $\mathcal{N}_\theta$ à approximer, à partir du vecteur $y_I \in \mathbb{R}^n$, le vecteur de coefficients $\bar{y}_I \in \mathbb{R}^{\bar{n}}$.



Entraînement du réseau de neurones

On veut donc entraîner le réseau de neurones $\mathcal{N}_\theta$ à approximer, à partir du vecteur $y_I \in \mathbb{R}^n$, le vecteur de coefficients $\bar{y}_I \in \mathbb{R}^{\bar{n}}$.



La loss minimisée pour l'entraînement est,

$$\mathcal{L}(\theta) = \frac{1}{N_s} \sum_{I=1}^{N_s} \|\bar{y}_I - \mathcal{N}_\theta(y_I)\|^2.$$



Un problème dans la méthode ?

Un problème potentiel se cache dans cette méthode :

- Nous entraînons le réseau de neurones $\mathcal{N}_\theta$ sur les projections **par rapport au p.s euclidien** des snapshots sur les bases V et $\bar{V}$.



Un problème dans la méthode ?

Un problème potentiel se cache dans cette méthode :

- Nous entraînons le réseau de neurones $\mathcal{N}_\theta$ sur les projections **par rapport au p.s euclidien** des snapshots sur les bases V et $\bar{V}$.
- Cependant, en phase online, le vecteur y obtenu grâce au ROM est le projeté sur V **par rapport au p.s issu de la forme bilinéaire symétrique** a_μ , c'est-à-dire,

$$\langle \cdot, \cdot \rangle_\mu = a_\mu(\cdot, \cdot) = \int_{\Omega} D_\mu(x, y) \nabla(\cdot) \cdot \nabla(\cdot) dx dy.$$



Un problème dans la méthode ?

Un problème potentiel se cache dans cette méthode :

- Nous entraînons le réseau de neurones $\mathcal{N}_\theta$ sur les projections **par rapport au p.s euclidien** des snapshots sur les bases V et $\bar{V}$.
- Cependant, en phase online, le vecteur y obtenu grâce au ROM est le projeté sur V **par rapport au p.s issu de la forme bilinéaire symétrique** a_μ , c'est-à-dire,

$$\langle \cdot, \cdot \rangle_\mu = a_\mu(\cdot, \cdot) = \int_{\Omega} D_\mu(x, y) \nabla(\cdot) \cdot \nabla(\cdot) dx dy.$$

- Ainsi, on entraîne le réseau de neurones $\mathcal{N}_\theta$ sur des données potentiellement **significativement différentes** de celles qu'il va rencontrer en phase online.



Un problème dans la méthode ?

Un problème potentiel se cache dans cette méthode :

- Nous entraînons le réseau de neurones $\mathcal{N}_\theta$ sur les projections **par rapport au p.s euclidien** des snapshots sur les bases V et $\bar{V}$.
- Cependant, en phase online, le vecteur y obtenu grâce au ROM est le projeté sur V **par rapport au p.s issu de la forme bilinéaire symétrique** a_μ , c'est-à-dire,

$$\langle \cdot, \cdot \rangle_\mu = a_\mu(\cdot, \cdot) = \int_{\Omega} D_\mu(x, y) \nabla(\cdot) \cdot \nabla(\cdot) dx dy.$$

- Ainsi, on entraîne le réseau de neurones $\mathcal{N}_\theta$ sur des données potentiellement **significativement différentes** de celles qu'il va rencontrer en phase online.
- C'est là que l'omission implicite de l'approximation $\approx$ de l'article devient problématique.



Cela se vérifie en pratique

Supposons que le réseau de neurones $\mathcal{N}_\theta$ soit parfaitement entraîné sur les données d'entraînement, c'est-à-dire qu'il produit exactement $\bar{y}_I$ pour chaque y_I .



Cela se vérifie en pratique

Supposons que le réseau de neurones $\mathcal{N}_\theta$ soit parfaitement entraîné sur les données d'entraînement, c'est-à-dire qu'il produit exactement $\bar{y}_I$ pour chaque y_I .

Soit $y_I = V^T(u_I - u_0)$ et $\bar{y}_I = \bar{V}^T(u_I - u_0)$ une paire d'entraînement.
L'approximation,

$$u_I \approx u_0 + Vy_I + \bar{V}\bar{y}_I,$$

est alors, par construction, aussi précise que possible sur V et $\bar{V}$ pour la norme $\|\cdot\|_2$.



Cela se vérifie en pratique

Supposons que le réseau de neurones $\mathcal{N}_\theta$ soit parfaitement entraîné sur les données d'entraînement, c'est-à-dire qu'il produit exactement $\bar{y}_I$ pour chaque y_I .

Soit $y_I = V^T(u_I - u_0)$ et $\bar{y}_I = \bar{V}^T(u_I - u_0)$ une paire d'entraînement. L'approximation,

$$u_I \approx u_0 + Vy_I + \bar{V}\bar{y}_I,$$

est alors, par construction, aussi précise que possible sur V et $\bar{V}$ pour la norme $\|\cdot\|_2$.

Cependant, en pratique, le ROM ne produit pas la projection de $y_I = V^T(u_I - u_0)$ sur V par rapport au p.s euclidien, mais par rapport à la forme bilinéaire symétrique a_μ qu'on appelle y_I^{ROM} , et,

$$y_I = V^T(u_I - u_0) \neq y_I^{ROM} = (V^T A_\mu V)^{-1} V^T A_\mu (u_I - u_0).$$



Cela se vérifie en pratique

On vérifie en pratique que l'erreur faite par les approximations,

$$(1) \quad u_0 + V y_I^{ROM},$$

et

$$(2) \quad u_0 + V y_I^{ROM} + \overline{V} \overline{y}_I,$$

sont globalement proches, le terme de correction non-linéaire $\overline{V} \overline{y}_I$ n'apportant aucun gain significatif en terme de précision.



Cela se vérifie en pratique

On vérifie en pratique que l'erreur faite par les approximations,

$$(1) \quad u_0 + V_{y_I}^{ROM},$$

et

$$(2) \quad u_0 + V_{y_I}^{ROM} + \overline{V}_{\overline{y}_I},$$

sont globalement proches, le terme de correction non-linéaire $\overline{V}_{\overline{y}_I}$ n'apportant aucun gain significatif en terme de précision.

Ainsi, même un réseau de neurones parfaitement entraîné sur les données d'entraînement ne peut pas corriger l'erreur faite par le ROM, car il n'est pas entraîné pour cela.



Erreur selon la reconstruction choisie

On compare l'erreur l^2 pour différentes reconstructions de u_I , en distinguant les projections issues du ROM et les projections euclidiennes :

u_{approx}	Erreur $l^2 \ \ u_I - u_{\text{approx}}\ _2$
$u_0 + Vy_I$	3.0×10^{-2}
$u_0 + Vy_I + \overline{V} \overline{y}_I$	2.0×10^{-3}
$u_0 + Vy_I^{\text{ROM}}$	3.3×10^{-1}
$u_0 + Vy_I^{\text{ROM}} + \overline{V} \overline{y}_I$	3.3×10^{-1}

Table: Résultats de l'erreur l^2 pour différentes reconstructions de u_I pour le cas non-affine II, avec $n = 50$ et $\bar{n} = 100$.

